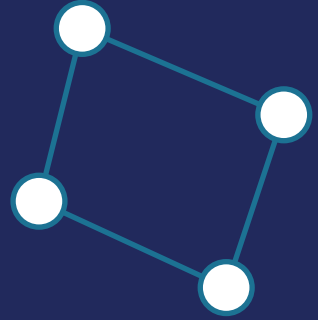


Graph Traversal

Representation, BFS, and DFS in C++

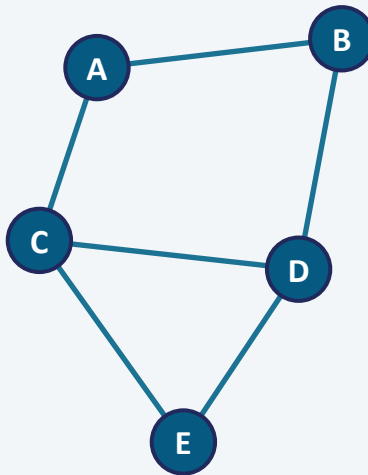


What is a Graph?

A graph $G = (V, E)$ is a set of **vertices (V)** connected by **edges (E)**.

- Vertices (nodes) represent items — e.g. cities, people, web pages
- Edges represent relationships or connections between vertices
- Directed graph: edges have a direction ($A \rightarrow B$)
- Undirected graph: edges have no direction ($A - B$)
- Weighted graph: edges carry a cost or distance value

Example Graph



$$V = \{A, B, C, D, E\} \quad E = \{AB, AC, BD, CD, DE, CE\}$$

Graph Components & Terminology

1

Vertex (Node)

A single point/entity in the graph

2

Edge

A connection between two vertices

3

Adjacent Vertices

Two vertices directly connected by an edge

4

Degree

Number of edges connected to a vertex

5

Path

A sequence of vertices connected by edges

6

Cycle

A path that starts and ends at the same vertex

Graph Representation: Adjacency Matrix

A 2D array of size $V \times V$. $\text{matrix}[i][j] = 1$ if an edge exists between vertex i and j , else 0.

- Simple to implement and check if an edge exists — $O(1)$
- Uses $O(V^2)$ space, even for sparse graphs (A sparse graph is a graph with relatively few edges compared with the maximum possible number of edges.)
- Symmetric for undirected graphs

```
int adjMatrix[MAX][MAX] = {0};

void addEdge(int u, int v) {
    adjMatrix[u][v] = 1;
    adjMatrix[v][u] = 1;    // undirected
}
```

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

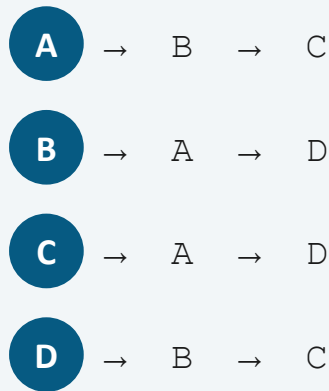
Graph: A-B, A-C, B-D, C-D

Graph Representation: Adjacency List

An array of lists. `adjList[i]` stores all vertices directly connected to vertex `i`.

- Space-efficient — $O(V + E)$, good for sparse graphs
- Checking if an edge exists takes $O(\text{degree}(v))$ in a typical list
- Commonly used for graph traversal algorithms

```
vector<int> adjList[MAX];  
  
void addEdge(int u, int v) {  
    adjList[u].push_back(v);  
    adjList[v].push_back(u); // undirected  
}
```



Graph: A-B, A-C, B-D, C-D

Breadth-First Search (BFS)

BFS explores a graph **level by level** — it visits all neighbors before moving to the next level.

- **BFS uses a Queue (FIFO — First In, First Out)**
- Start at a source vertex, mark it visited, and enqueue it
- Dequeue a vertex, visit it, then enqueue all its unvisited neighbors
- Repeat until the queue is empty
- Used for: shortest-path distances in unweighted graphs, level-order traversal, and connected-component discovery

Queue Behavior (FIFO)

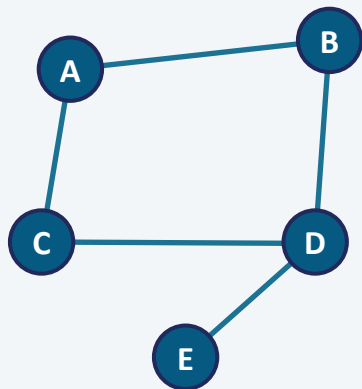


front → dequeue next rear → new items added here

Steps:

- Enqueue start vertex, mark visited
- Loop: dequeue front vertex, print it
- Enqueue its unvisited neighbors, mark visited
- Stop when queue is empty

BFS Example — Step by Step



Start BFS from A

Visit order: A → B → C → D → E

Step	Action	Enqueue neighbors
1	Visit A	B, C
2	Visit B	D
3	Visit C	(D already queued)
4	Visit D	E
5	Visit E	—

Queue contents over time

```
Start:      [A]
Visit A →   [B, C]
Visit B →   [C, D]
Visit C →   [D]
Visit D →   [E]
Visit E →   [ ] (done)
```

BFS Implementation in C++

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;
const int MAX = 100;
vector<int> adjList[MAX];
bool visited[MAX];

void bfs(int start) {
    queue<int> q;
    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        int current = q.front();
        q.pop();
        cout << current << " ";

        for (int neighbor : adjList[current]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
}

int main() {
    int vertices = 5, edges = 5;
    int u, v;

    cout << "Enter " << edges << " edges (u
v):\n";
    for (int i = 0; i < edges; i++) {
        cin >> u >> v;
        adjList[u].push_back(v);
        adjList[v].push_back(u);    // remove
for directed graph
    }

    cout << "BFS starting from vertex 0: ";
    bfs(0);

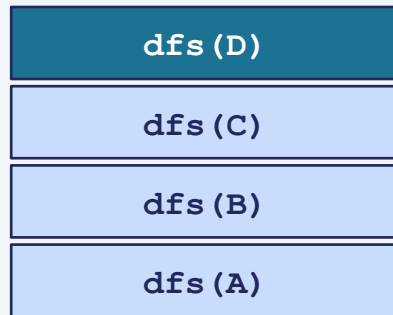
    return 0;
}
```

Depth-First Search (DFS)

DFS explores as **far as possible** along one branch before backtracking.

- **DFS uses recursion (or an explicit Stack — LIFO)**
- Start at a source vertex and mark it visited
- Recursively visit an unvisited neighbor, going deeper
- When no unvisited neighbor remains, backtrack (return from recursion)
- Used for: cycle detection, topological sorting, maze/path solving, and connected components

Call Stack Behavior (LIFO)

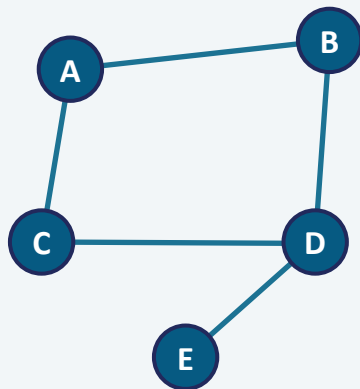


top → returns (backtracks) first

Steps:

- Call `dfs(start)`, mark it visited
- Visit an unvisited neighbor by recursively calling `dfs` on it
- When a vertex has no unvisited neighbor, the call returns (backtrack)
- Continue until the first call finishes

DFS Example — Step by Step



Start DFS from A (same graph)

Visit order: A → B → D → C → E

Recursive call trace

```
dfs(A)  visited={A}
  → dfs(B)  visited={A,B}
    → dfs(D)  visited={A,B,D}
      → dfs(C)  visited={A,B,D,C}
        (C's neighbors A,D visited)
        backtrack to D
      → dfs(E)  visited={A,B,D,C,E}
        backtrack to D, then B, then A
    dfs(A) finished — all vertices visited
```

DFS Implementation in C++

```
#include <iostream>
#include <vector>
using namespace std;

const int MAX = 100;
vector<int> adjList[MAX];
bool visited[MAX];

void dfs(int current) {
    visited[current] = true;
    cout << current << " ";

    for (int neighbor : adjList[current]) {
        if (!visited[neighbor]) {
            dfs(neighbor);    // recursive call
        }
    }
}
```

```
int main() {
    int vertices = 5, edges = 5;
    int u, v;

    cout << "Enter " << edges << " edges (u
v):\n";
    for (int i = 0; i < edges; i++) {
        cin >> u >> v;
        adjList[u].push_back(v);
        adjList[v].push_back(u);    // remove
for directed graph
    }

    cout << "DFS starting from vertex 0:
";
    dfs(0);

    return 0;
}
```

BFS vs DFS • Graph vs Tree

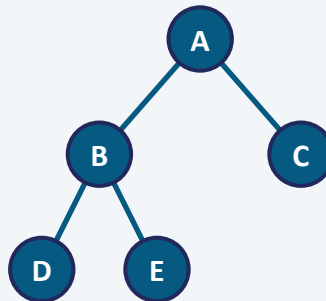
BFS vs DFS

Aspect	BFS	DFS
Data structure	Queue	Recursion / Stack
Order	Level by level	Goes deep first
Memory use	Can be higher (wide graphs)	Can be higher (deep graphs)
Shortest path	Yes, in unweighted graphs	Not guaranteed
Typical use	Shortest path, level order	Cycle detection, topological sort

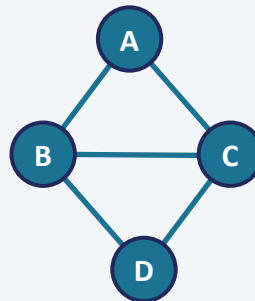
Graph vs Tree

- A tree is a connected graph with no cycles; therefore, there is exactly one simple path between any two nodes
- A tree has $N-1$ edges for N nodes; a general graph can have a different number of edges
- A tree can be rooted for a parent-child hierarchy; a general graph has no required root or hierarchy
- Every tree is a graph, but not every graph is a tree

Tree (no cycles)



Graph (has a cycle)



In an undirected graph, BFS/DFS needs a visited array to avoid processing already-seen vertices. A tree has no cycles, but the parent can still appear as an already-seen neighbor in an adjacency list.